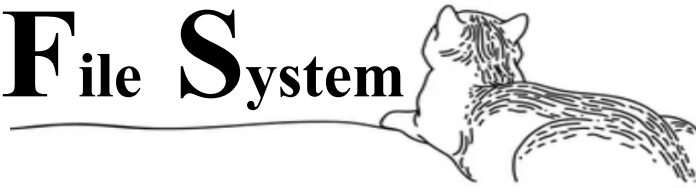
CATS File System



Names: Milo Pesce Ares, Allie Young, Rosa Melton, Abha Deshpande, Patricia Ann Payumo
IDs: MPA 921926821, AY 923129607, RM 921337778, AD 923314454, PAP 923555591
Github Name: milo-pa
Github Link: https://github.com/CSC415-2026-Spring/csc415-filesystem-CATS
Team Name: CATS
Term: Spring 2026
Section: 3

Table of Contents



- **CATS File System..... 1**
- **Table of Contents..... 2**
- **Project Description..... 3**
- **Plan/Approach.....4**
  - Planning:..... 4
  - Drawing Out Approach:.....6
- **Shell Commands..... 10**
- **Issues Encountered.....11**
- **Function Descriptions.....17**
- **Contributions..... 21**
- **Screenshots..... 24**
  - Compilation:..... 24
  - Execution:..... 25
- **Milestones 1 and 2 Write-Ups.....36**
  - VCB Struct Description..... 36
  - Free Space Struct Description.....37
  - Root Directory Description.....38
  - Team Coordination.....39
  - HexDump.....41

Project Description 

This is a custom file system. It is an extent-based system built upon a simulated block storage system. It operates using Logical Block Addressing (LBA) read and write operations. The Volume Control Block (VCB) is stored at 0 and contains significant metadata about the file system such as the total number of blocks, block size, location of the freespace and root directory. When the file system is initialized, the VCB is loaded into memory and used to manage disk operations.

Free space management is handled using an extent table separate from the VCB. Extents are assigned to files instead of allocating individual blocks to reduce fragmentation and performance (minimizing disk accesses to data). Each file and directory maintains an extent list which describes where its data is on disk. Free space is managed by using a separate extent table which is stored on disk and is loaded into memory during initialization. This table tracks all available blocks and when a space request is made, the file system searches for a sufficiently sized extent, allocates the required blocks, and updates the free space. After files or directories are deleted, their memory is freed and the blocks are returned to the free space system and merged when possible to reduce fragmentation.

Directories are implemented as arrays of directory entries, where each entry stores metadata. Each directory includes ".", and ".." which allows hierarchical navigation. Path traversal is handled via a parsing function which resolves both absolute and relative paths. File operations are implemented through buffered IO and support the flags: readonly, writeonly, readwrite, create, truncate, append. File operations use File Control Blocks (FCBs) to track open files, current position, and buffer state. Reducing direct disk access via buffering improves performance since it allows partial reads and writes in a block. We have a provided shell to test and demonstrate the functionality of the file system which supports various commands for displaying the contents of files and directories, creating and deleting files and directories, copying and moving files, and directory navigation.

PLEASE NOTE: We have a debug macro which prints various debugging statements from each function, essentially acting as a stack trace so we can follow what is happening and verify that things are being updated correctly throughout the flow of execution. This can be tested with `make clean` then `make run DEBUG=1`

Plan/Approach



Planning:

Everything is planned around three milestones as defined in the project description. We spent the longest time on milestone 1 because it was the foundation in which we established the core structures: VCB, freespace management, and block management for directory creation (and later file creation). This approach allowed us to build a solid foundation before proceeding on other features to ensure a progressive working expansion.

Milestone 1:

Milestone 1 was initializing and formatting the filesystem and designing the VCB (Volume Control Block). The VCB stores block count, block size, root directory location, and free space location. To allow the file system to reload its state correctly after restarting, we are storing the locations of the root directory and free space structure in the VCB. We also thought about the order of execution: the freespace system is initialized before the root directory is made since making a directory is dependent on block allocation. The goal of this milestone was having a volume that could be initialized, preserving metadata across multiple runs, allocating space for the root directory, and having a starting point for the file system hierarchy.

Milestone 2:

Milestone 2 focused on user interaction through shell commands. This phase we focused on implementing a bunch of helper functions such as parsePath, loadDir, writeDirectory, findDirEntry, and findUnusedDirEntry. A significant design decision was to centralize path traversal via parsePath() instead of making multiple path traversals in every command. Another design choice was storing directories contiguously on disk. This makes loading directory and writing logic more efficient, since directory entries could be read in a single operation without needing to traverse multiple extents. We also implemented current working directory tracking, which retains state in memory and ensures consistency across operations, and a function growFile() to make sure directories can expand when new entries are added that cannot fit in the already allocated space (50 entries by default).

Milestone 3:

Milestone 3 focused on completing the file system by implementing file operations via buffered I/O and more nuanced extent management. The primary goal of this phase was to extend the system to support files. Helper logic such as getNextExtent() was introduced to allow file data to be read and written across multiple extents.

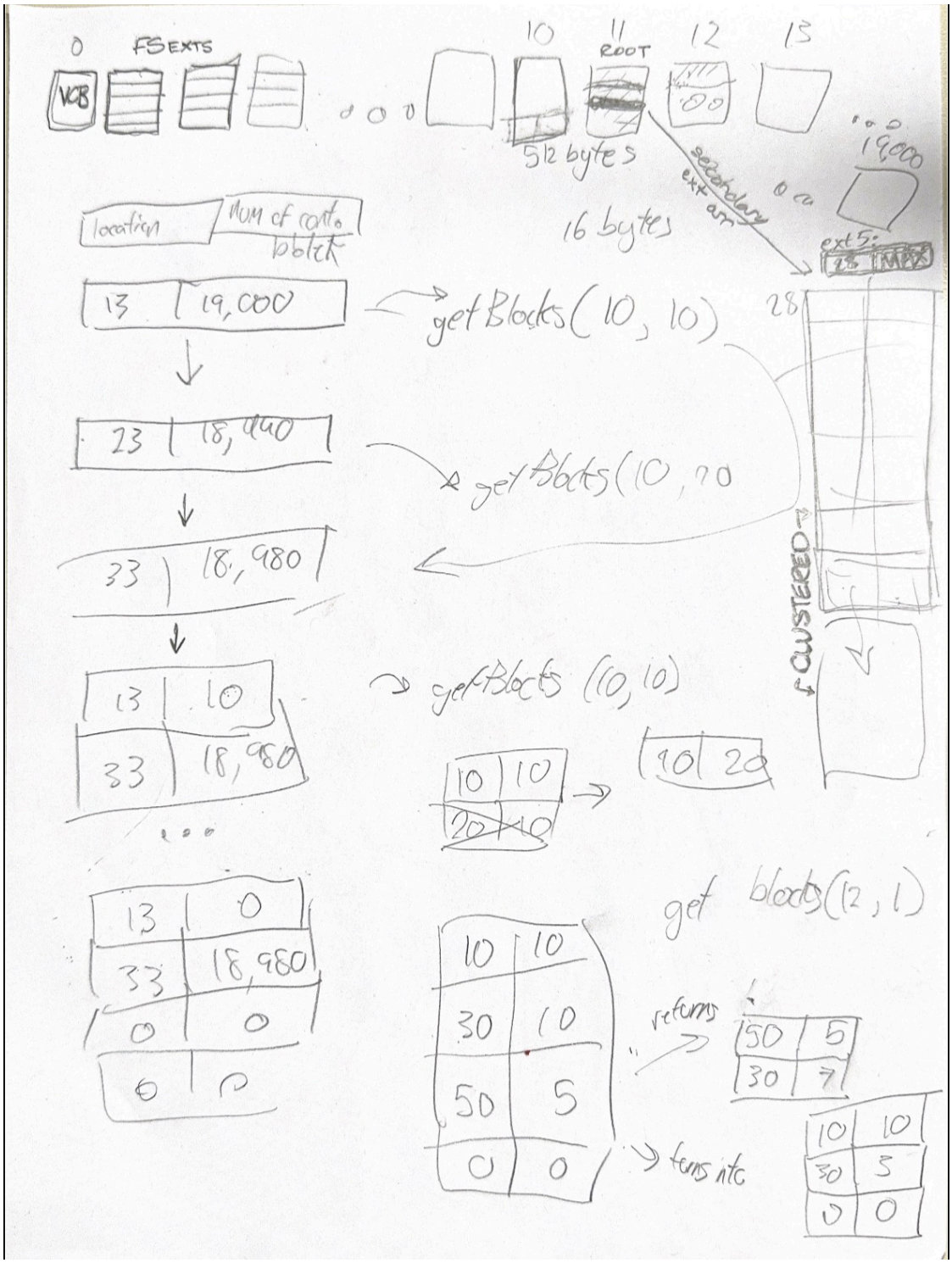
In this phase, we worked on implementing the buffered I/O interface, including the functions b_open, b_read, b_write, b_seek, and b_close. These functions are responsible for managing open files through File Control Blocks (FCBs), tracking file offsets, and handling buffered access to disk. We also planned support for standard file access flags such as create, truncate, append, read-only, write-only, and read-write. Managing file offsets and buffer state was an important part of this design, especially ensuring that buffers are invalidated when moving across blocks and that file position is updated correctly across operations.

Another key aspect of this milestone was handling file growth and space allocation. This included allocating additional blocks when writing beyond the current capacity of a file and ensuring that newly allocated space is properly reflected in the file's extent list. Reads and writes must correctly traverse these extents to maintain data consistency. We also made improvements to the freeBlocks() function, adding sorting on insertion so that merging adjacent extents was more efficient.

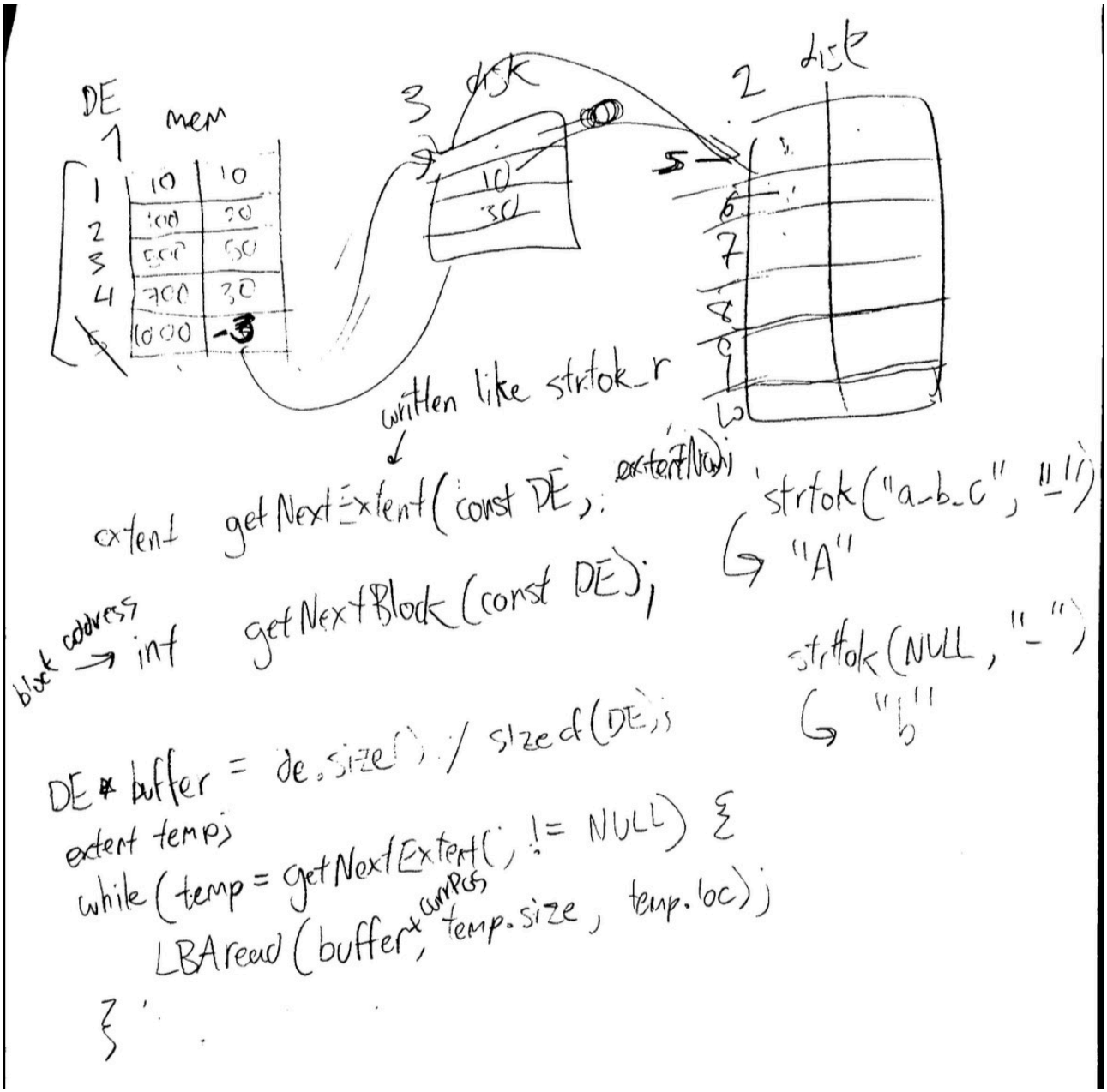
Since this is the final milestone, testing is significant in validating the system. We focused on verifying that shell commands behave correctly, especially those dependent on file operations such as cp, mv, and cat. Additional testing included edge cases such as large files, fragmented storage, and ensuring consistency between in-memory structures and on-disk data which we tested via the hexdumps. Improving error handling by writing more informative debug statements was also an important part of this phase.

Drawing Out Approach:

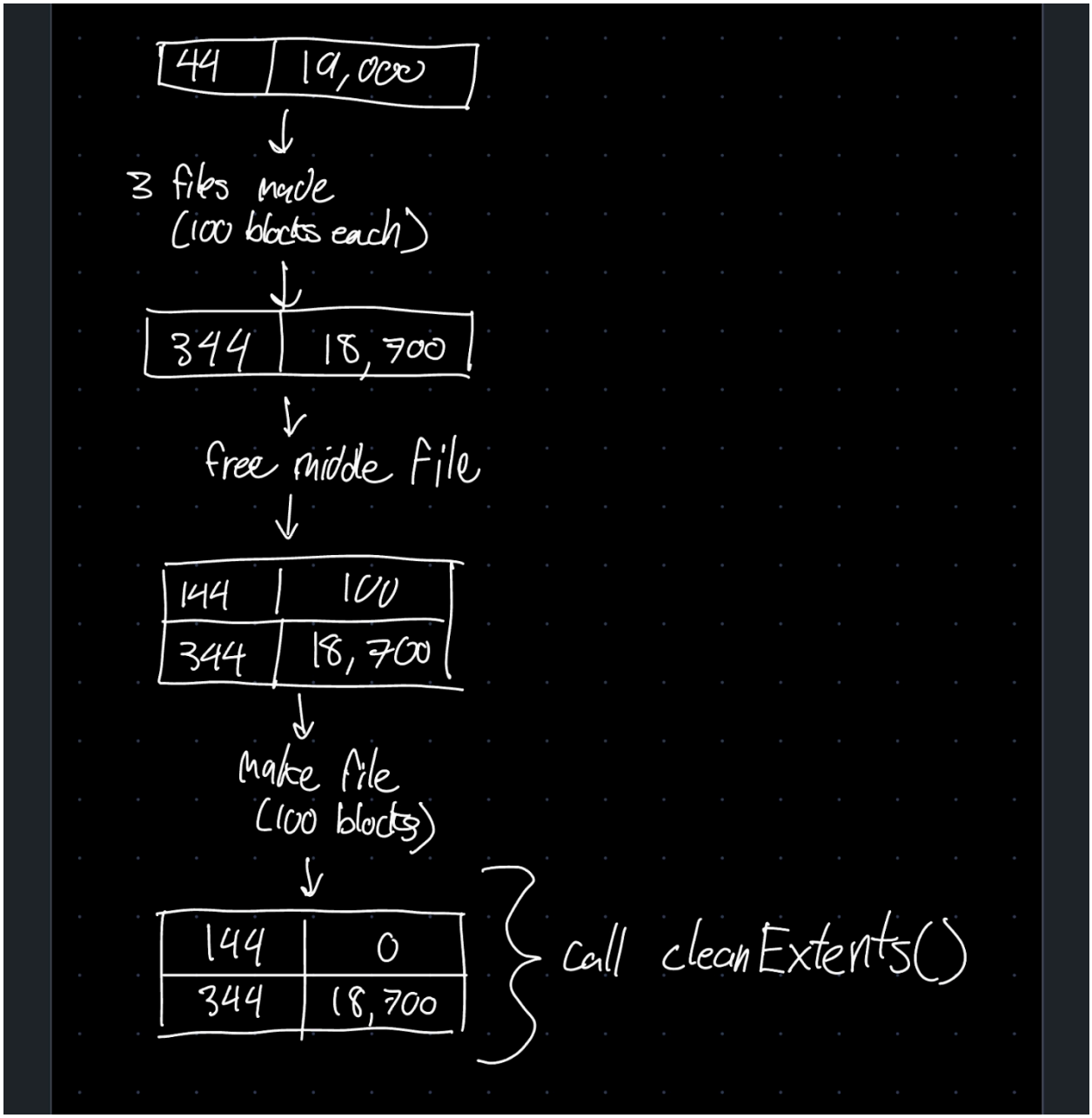
04/09 - Mapping out getBlocks() interaction with free space (left half) and secondary extent table handling with unsigned int sentinels (right half)



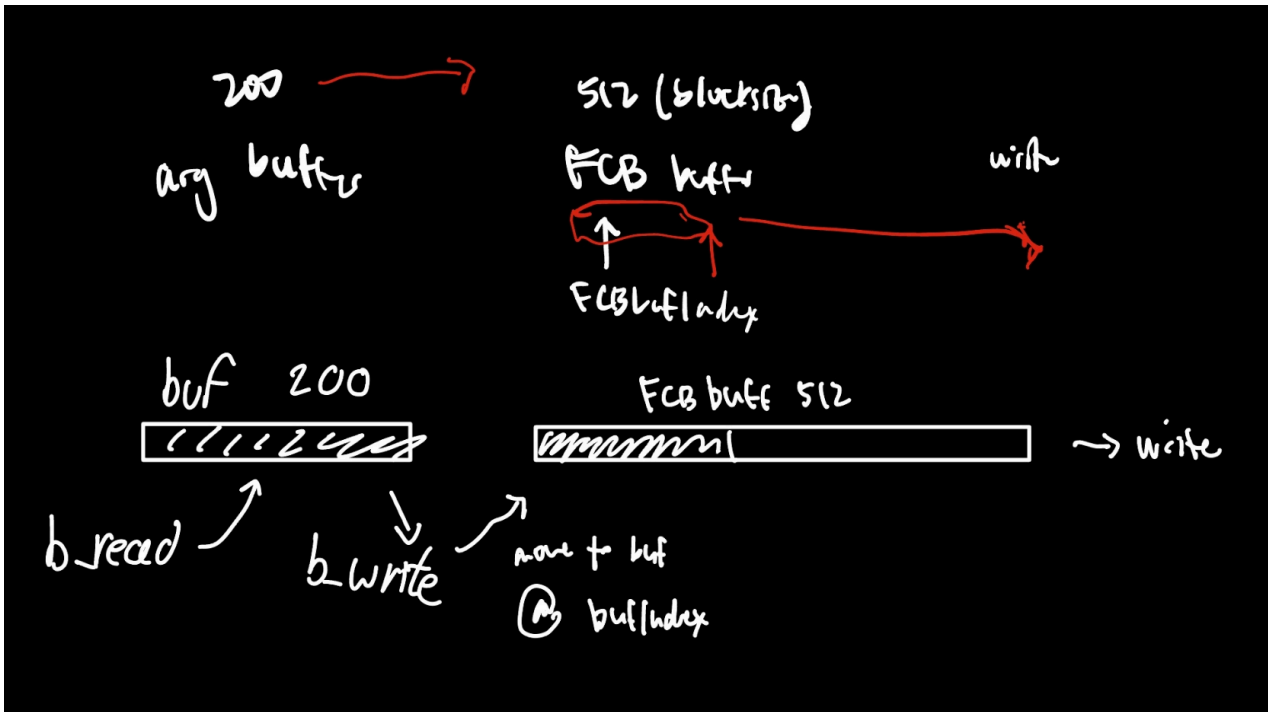
04/23 - Initial mapping out of getNextExtent() to handle the logic of traversing through a directory entry's array of extents; initial plan was to follow the logic of strtok_r() with a 'save position' as the last index returned



Drawing for when we would need to call cleanExtents() in getBlocks() and freeBlocks()



Drawing of b_write() logic



Shell Commands 

Command	Status	Description	Limitations
ls	☑	list directory contents	No issues (that we know of)
cd	☑	change current working directory	Cannot cd directly to / from a nested directory (must use cd ../..)
pwd	☑	show current working directory	No issues (that we know of)
md	☑	create new directory	No issues (that we know of)
rm	☑	remove file or directory	No issues (that we know of)
touch	☑	create new file	No issues (that we know of)
cat	☑	show file contents	Works for files of any size as long as they only have primary extents (5)
mv	☑	make a file copy, delete original	Works for files of any size as long as they only have primary extents (5)
cp	☑	make a file copy within FS	Works for files of any size as long as they only have primary extents (5)
cp2l	☑	file copy from FS to Linux	Works for files of any size as long as they only have primary extents (5)
cp2fs	☑	file copy from Linux to FS	Works for files of any size as long as they only have primary extents (5)
help	☑	show commands available	works as intended
history	☑	show command history	works as intended
exit	☑	exit shell	works as intended

Issues Encountered



MILESTONE 1

Issue # 1

- ➔ **Problem:** had to remove root dir length in VCB, can just read 1 block
- ➔ **Description:** we intended to store the length of the root directory within our VCB so we could call LBAread() with the length as the first parameter, but didn't know how to properly get this length.
- ➔ **Resolution:** Our workaround was to use at least 1 block to feed into LBAread()
  (Issue # 11 ties into this as well)

Issue # 2

- ➔ **Problem:** getBlocks only searches for contiguous space when we want discontiguous extents for files
- ➔ **Description:** A discussion frequently brought up but have not encountered an issue yet with this design so TBD…
- ➔ **Resolution:** Based on feedback in office hours, we changed getBlocks() to return an array of extents instead and added a second parameter for a minimum number of contiguous blocks to be used only when creating directories since we know we want those stored contiguously.

Issue # 3

- ➔ **Problem:** Freespace extents array was inside VCB
- ➔ **Description:** This prevented us from accessing getBlocks() during runtime, which meant we would have to load from disk every time we wanted to call it.
- ➔ **Resolution:** Had to be moved out; storing the starting block of free space location is satisfactory for the rest of our implementations.

Issue # 4

- ➔ **Problem:** Had to pass volume control block to everything
- ➔ **Description:** What the problem says; had a repeated, unnecessary parameter that could be improved.
- ➔ **Resolution:** We are using a global instance of the volume control block so it can be accessed across many files and functions and because there is only one per volume

Issue # 5

- ➔ **Problem:** Had freespace extents global when it did not need to be
- ➔ **Description:** Turned out to be unnecessary implementation. Simple removal
- ➔ **Resolution:** Removed the extern declaration. Freespace extents never need to be accessed by external functions since we have getBlocks

Issue # 6

- ➔ **Problem:** Currently not supporting secondary/tertiary extent tables
- ➔ **Description:** We are not accounting for extents greater than the 10 initialized with the file system, and we will be in trouble if we do not handle this as we write more and more to our volume…
- ➔ **Resolution:** Since this milestone is only focused on initializing everything, these TODOs will be pushed as we fill our volume in the next milestone. Office hours feedback - we will not need tertiary extent tables at all given the scope of this project.

Issue # 7

- ➔ **Problem:** Entire file deleted
- ➔ **Description:** N/A
- ➔ **Resolution:** `git revert`

Issue # 8

- ➔ **Problem:** Memory leak that was incredibly difficult to track
- ➔ **Description:** We were trying to populate fsInit.c with all the completed and compiling files that we had finished. However, we started experiencing inexplicable, intermittent errors. Print statements wouldn't run in order, it changed from machine to machine, and seemingly triggered randomly. Classic C stuff that everyone loves
- ➔ **Resolution:** A buffer that was not evenly divisible by block size was being passed to LBARead(). We think this was the root cause, but with intermittent memory problems you can never be too sure. This led us to include the file and function names in all error messages to help with future tracking.

Issue # 9

- ➔ **Problem:** handling "." and ".." for root directory
- ➔ **Description:** Special case since ".." should point to the parent directory, but for the root directory, there's technically no parent.
- ➔ **Resolution:** In createDirectory, if the parent pointer is NULL, the new directory is treated as the root and ".." is set to point back to itself.

Issue # 10

- ➔ **Problem:** safely initializing unused directories

- ➔ **Description:** if directory memory isn't cleared first, unused entries may contain garbage values which may cause issues when checking if an entry is free or used
- ➔ **Resolution:** decided to replace malloc+memset with calloc() instead to ensure the unused entries are empty (less calls the better; calloc() appropriate for this use);

Issue # 11

- ➔ **Problem:** initFreeSpace() and initVCB() relied on the other to already be initialized and could not resolve without hardcoding in these function calls…
- ➔ **Description:** initVCB required free space location to already be set in initFreeSpace, yet initFreeSpace relied on an already instantiated member of VBC.
- ➔ **Resolution:** Remove initVCB() and assign VCB's members as we call its respective initialization functions (ie createDir() for root, initFreeSpace()

Issue # 12

- ➔ **Problem:** VCB's buffer malloc'd with less than blockSize causing LBAread() issues
- ➔ **Description:** We didn't want to malloc an entire block to be occupied by only 32 bytes for VCB and we are stubborn
- ➔ **Resolution:** There is no way around this we need a block for LBAread to work so we applied the 'blocks needed' formula provided from lecture and malloc'd that to VCB

  "It's okay to waste a little bit."

Issue # 13

- ➔ **Problem:** Making header compiled debug print function (Milo)
- ➔ **Description:** The file system shouldn't print messages, but we need print statements for debugging. After going to office hours and asking about this, I was told I could add a debug.h file that used macros to define a function that could be called for printing debug statements. However, Bierman warned me that it could be a little tricky, particularly with creating a function that takes in an indeterminate amount of params.
- ➔ **Resolution:** I looked online first for implementation examples on passing multiple parameters to a function and debug macro functions. Here are some of the links that I found helpful:
  - ◆ https://ilostmynotes.blogspot.com/2011/04/ifdef-debug-print-statements-in-c.html
  - ◆ https://sqlpey.com/c/c-cpp-debugging-macros/

  I then added the header file to the makefile and added a simple command to simplify calling it

MILESTONE 2

Issue # 14

- ➔ **Problem:** We were not allocating space for sentinel extent in the case getBlocks returned the max allocated (numblocks/mincontiguous)
- ➔ **Description:** Since there is no way to know how many extents will be needed, we allocated for worst case, but not worst case + 1
- ➔ **Resolution:** We did a realloc if there was no allocated space for the sentinel

Issue # 15

- ➔ **Problem:** Who should handle logic for freeing parsePathInfo struct?
- ➔ **Description:** Certain members only need to be freed on certain conditions, i.e parentDir only needs to be freed if it is not the rootGlobal or cwdGlobal. The callers of parsePath should not be responsible for this logic
- ➔ **Resolution:** Helper freePPI() which the callers just need to make sure to call after using parsePath

MILESTONE 3

Issue # 16

- ➔ **Problem:** When running ls, we were getting an invalid path error
- ➔ **Description:** First command tested was ls so it exposed all the errors we had, not necessarily ones related to ls specifically.
- ➔ **Resolution:** The issue was that the cwd path buffer was never initialized to root “/”

Issue # 17

- ➔ **Problem:** We needed to handle root case in open and closedir
- ➔ **Description:** Do we load a copy, or just use the globals and write a check in close so our global loaded root dir and cwd are not accidentally freed?
- ➔ **Resolution:** We are loading a copy of root to avoid having to make exceptions in both funcs

Issue # 18

- ➔ **Problem:** readdir was not stopping/checking for last entry
- ➔ **Description:** This was a simple code mistake
- ➔ **Resolution:** Added in check for last entry

Issue # 19

- ➔ **Problem:** Memleak – cd

- ➔ **Description:** There was a memleak when cd-ing from a non root directory
- ➔ **Resolution:** Memory was not being freed correctly in fs_setcwd

Issue # 22

- ➔ **Problem:** Memleak – ls -l
- ➔ **Description:** There was a memleak when using ls -l
- ➔ **Resolution:** Memory was not being freed correctly in fs_stat

Issue # 23

- ➔ **Problem:** we are not tracking size of ext tables
- ➔ **Description:** This is needed for functions like growFile
- ➔ **Resolution:** create helper function that reads block by block and returns a counter of each read, stopping at sentinel value

Issue # 24

- ➔ **Problem:** Cat not working for copied files
- ➔ **Description:** We hardcoded a file to modularly test cat / b_read() without reliance on b_write(), and cat worked fine. However when we copy a file, whether with cp or cp2fs, cat will not print the contents of the file. Cp2l seems to be working fine – contents printing in linux vm.
- ➔ **Resolution:** b_write() was not getting new extents for files

Issue # 25

- ➔ **Problem:** Copied files had content written, but cat was not printing them
- ➔ **Description:** We checked that cp was working with the hexdumps, but cat was not printing the contents of copied files. When checking with hexdumps, we noticed that the size was inflated
- ➔ **Resolution:** growFile() was updating the size of a file when allocating for it, when the file was not actually being written to. This broke b_read()

Issue # 26

- ➔ **Problem:** Similar to issue #15, getNextExtent() callers have to free only if secondary/tertiary extents returned
- ➔ **Description:** This logic should not be the responsibility of the caller, can cause invalid frees or accidental freeing of primary extents
- ➔ **Resolution:** First we created a helper freeExt() to only free extent pointer if its extent array contains either tertiary or secondary sentinel value, decided to instead change getNextExtent() to always return heap-allocated so that callers just consistently free

Issue # 27

- ➔ **Problem:** When writing to a file, b_write() can allocate 100 blocks in one extent
- ➔ **Description:** If the user does not actually write 100 blocks, we have to split the extent and free the unused blocks
- ➔ **Resolution:** We have a trim() function which does that

Issue # 28

- ➔ **Problem:** b_read() was not working for files above 200 bytes
- ➔ **Description:** It was resetting the tracking variables for each call. The shell calls b_read with a buffer of 200 bytes
- ➔ **Resolution:** Rewrote to support sequential calls, added tracking variable updates in b_seek() and b_open() if in APPEND mode

Issue # 29

- ➔ **Problem:** We could not iron out the bugs in getNextExtent() for secondary/tertiary traversal in time
- ➔ **Description:** Our commands support files of any size, as long as they are stored in primary extents (of which we have 5 slots per entry by default). As long as the disk is not super fragmented, when we create and grow a file, it allocates one extent where possible. So this should not be an issue unless the disk is very fragmented.
- ➔ **Resolution:** Could not fix in time.

Function Descriptions 

File I/O - b_io.c

- ➢ b_open
  - This function is responsible for locating a file within the directory struct and initializing a file control block for that file. It supports several flags: O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_TRUNC, O_APPEND

- ➢ b_read
  - This function reads data from a file into a user buffer. It retrieves data from file's extents. The function maintains the file pos across multiple calls, allowing sequential reads.

- ➢ b_write
  - The function writes data from a buffer into a file. It handles allocating new blocks when needed and updates file's metadata. It works with our extent storage system to determine where data should be written and uses buffering.

- ➢ b_seek
  - This function updates the file pos based on the specified mode (SET/CUR/END). Making sure that the position isn't negative because that's invalid. If the new pos falls into a different disk block that its prev pos, the file buffer invalidates to make sure the new accesses correct data.

- ➢ b_close
  - This function closes an open file descriptor and frees the associated FCB entry. Once a file closes, no further operations can be done using that descriptor, and helps maintain proper resource management.

Directory functions - fsDir.c

- ➢ createDirectory

- This function allocates and initializes a new directory structure. It calculates needed memory, allocates contiguous blocks for free space system, and initializes directory entries including “.” and “..”.

➢ writeDirectory
- This function writes directory content to disk.

➢ loadDir
- This function loads a directory from disk into memory. It reads the extents and reconstructs the entire directory structure, giving access and modified in memory.

➢ findDirEntry / findUnusedDirEntry
- This function searches through directory entries. findDirEntry locates an entry by name, findUnusedDirEntry finds the first empty slot. If no space is available, the directory is expanded.

Functions called by shell - mfs.c

➢ fs_mkdir
- This function creates a new directory within the filesystem. It initializes “.”, and “..” entries, and it also updates parent directory to include new directory entry which ensures proper hierarchical structure of the directory.

➢ fs_rmdir
- This function removes a directory, but only if its completely empty. Only having “.” and “..” entries defines an empty directory. It frees all the allocated blocks related with the directory and removes its entry from the parent.

➢ fs_opendir
- This function opens a directory to be read. It loads the directory entries into memory and a directory descriptor tracks the curr pos within the directory and iterates through the contents of the directory.

➢ fs_closedir
- This function closes an open directory descriptor. It frees any memory allocated for the directory to make sure that resources are properly released after traversal is complete.

- fs_readdir
  - This function reads the next entry that's valid from an open directory. It iterates through skipping unused or empty directory entries, and then returns meta data only for valid entries.

- fs_setcwd
  - This function changes the current working directory, using parsePath to locate the directory targeted, and updates the global directory pointer so new operations are updated to the new location. Implements a stack in order to clean up path.

- fs_getcwd
  - This function returns the current working directory as a string which copies the internal path into a user buffer.

- fs_isFile
  - This function checks whether a given path refers to a regular file.

- fs_isDir
  - This function checks whether a given path refers to a directory.

- fs_delete
  - This function removes a file, and frees the file's allocated blocks and marks its directory entry as unused, which 'deletes' the file.

File System initialization - fsInit.c, fsVCB.c, fsStorage.c

- initFileSystem
  - This function loads the VCB, and checks whether the volume is formatted. If it isn't, it initializes free space, creates the root directory, and writes needed metadata to disk.

- exitFileSystem
  - This function handles cleanup when the system is shutdown, freeing allocated memory, making sure no mem leaks occur and the system is clean.

- loadVCB

- This function reads the VCB from disk into memory, allowing access to its metadata.

➢ initFreeSpace
- This function initializes the free space tracking system by creating an extent table and writing it to disk.

Supporting

➢ parsePath
- This function processes a given file path. It determines the parent directory, the entry index chosen, and the name of the final element; all of these are essential for navigating the file system.

➢ DEBUG
- This function is used throughout the file system to assist with troubleshooting. When it is enabled, it prints formatted messages to standard error including the function name and line number where it came from.

Free Space - fsStorage.c

➢ getNextExtent
- This function iterates through extents of a file or directory. It returns the next valid extent, which allows sequential access to all blocks associated with that entry.

➢ growFile
- This function expands a file or directory by allocating additional disk blocks and updating the extent list accordingly. It requests new blocks from the freespace system and inserting them into the files existing extent.

➢ getBlocks
- This function allocates blocks from the freespace system. It returns a list of extents that represent the allocated space.

➢ freeBlocks
- This function releases previously allocated blocks and returns it to the free space system. It merges adjacent free extents to reduce fragmentation.

Contributions



- A table of who worked on which components

Allie	- Conceptual charts/outline - Reformat structs for padding - Add in VCB, extent, directory entry structs - Remove freespace from VCB, replaced with block location - Add root dir length in VCB - VCB function signatures - Implement loadVCB() - Implement initVCB() - Implement isVCBOurs() - getBlocks signature - freeBlocks signature - Implement initFreespace() - Implement getBlocks() - Implement freeBlocks() - Remove initRootDir(), can just use createDir() - Rewrote getBlocks() to return array of extents - Implement fs_opendir() - Implement fs_readdir() - Implement fs_closedir() - Handle root exception in fs_opendir() - Add last entry check in fs_readdir() - Implement loadDir() - Set up root dir and cwd globals - Implement parsePath() - Implement freePPI() - Bug fixes in fs_getcwd() - Bug fixes in fs_readdir() - Test ls command and fix all issues - Test md, rm, cd, pwd - Implement b_open() - Implement b_close() - Test touch - Add more informative debug prints in every function (essentially stack trace) + print cwd for prompt if debug macro on - Wrote helper formatTime() to format times for debug print - Rewrote freeBlocks() to sort on insertion - Wrote hardcoded data into a file to test cat (b_read() w/o b_write()) - Fix bugs in b_io() - Test cat, mv, cp, cp2fs, cp2l

	- Rewrote b_read() to work for sequential reads - Update tracking variables correctly in b_seek() and if in append mode
Milo	- Designing directory entry struct - Designing volume control block struct - function signatures for free space - Lots of bug testing and code review - function signatures in fsVCB.h - Dividing group work - Checking for VCB init and instantiating in fsInit.c - File organization/architecture, naming, structs and calls - VCB architecture with globals and function signatures - Fixing major memory leak that was in loadVCB() - Restructuring usage of VCBGlobal across all files & fsInit.c - Adding Debug helper file and all debug comments - Updated comments on all files - Reviewed Hexdumps - Function signature for getNextExtent() - Implemented fs_opendir() - Implemented fs_readdir() - Implemented fs_closedir() - Implemented findDirEntry() - Implemented findUnusedDirEntry() - Implemented writeDirectory() - Incorporating system architecture into fsInit.c - Implemented b_write() - Added helper function getNextBlock() - Added helper function seekCursor() - Implemented trim()
Rosa	- formatting and verifying function calls in fsInit.c - Minor edits + additions of error messages for debugging in all files (to be replaced by debug header) - Edit of allocation of buffer in loadVCB for LBAread - Filled in descriptions and resolutions of Issues - Directory && VCB struct Descriptions - Team Coordination section of write-up - created file dedicated to helpers handling extent logic, fsStorageExts.c - implemented checkSentinel() - implemented findFirstFree() - implemented getTableSize() - implemented getNextExtent() - implemented createSecondary() - implemented growFile() in fsDir.c - added defines for sentinels in fsStorage.h

	- implemented freeExt() → DEPRACATED, changed getNextExtent() to account for proper free() - implemented printExts() for debugging
Trish	- Format writeup - directory system (fsDir.c && fsDir.h) - createDirectory implementation - root directory initialization (“.” && “..”) - scaffolded fsInit - Directory && VCB struct Descriptions - Project Description of write-up - Outlined and documented Plan/Approach across 3 Milestones - Function Descriptions of write-up - Contributed to Shell Commands table - implemented fs_delete - implemented fs_rmdir - implemented b_seek() - implemented cmd_mv - implemented cmd_cp
Abha	- HexDump analysis - Added bool check for freespace initialization - Implemented fs_getcwd() - Implemented fs_setcwd() - Implemented fs_mkdir() - Implemented b_read() - Implemented fs_stat() - Testing commands cd, pwd, history and help - Execution screenshots in the writeup - Compilation screenshots

Screenshots



Compilation:

```
~/Documents/csc415-filesystem-CATS  main !1 ?1
23:43:02
❯ make

gcc -c -o fsshell.o fsshell.c -g -I.
gcc -c -o b_io.o b_io.c -g -I.
gcc -c -o fsDir.o fsDir.c -g -I.
gcc -c -o fsStorage.o fsStorage.c -g -I.
gcc -c -o fsInit.o fsInit.c -g -I.
gcc -c -o fsVCB.o fsVCB.c -g -I.
gcc -c -o fsDirSearch.o fsDirSearch.c -g -I.
gcc -c -o mfsCwd.o mfsCwd.c -g -I.
gcc -c -o mfsDirItr.o mfsDirItr.c -g -I.
gcc -c -o mfsGod.o mfsGod.c -g -I.
gcc -c -o mfsInfo.o mfsInfo.c -g -I.
gcc -c -o fsStorageExts.o fsStorageExts.c -g -I.
gcc -o fsshell fsshell.o b_io.o fsDir.o fsStorage.o fsInit.o fsVCB.o fsDirSearch.o mfsCwd.o mfsDirItr.o mfsGod.o mf
sInfo.o fsStorageExts.o fsLow.o -g -I.  -lm -l readline -l pthread

~/Documents/csc415-filesystem-CATS  main !1 ?1
```

Execution:

Commands: cd and pwd

```
student@student:~/Documents/csc415-filesystem-CATS$ valgrind --leak-check=full ./fsshell SampleVolume 10000000 512
==74322== Memcheck, a memory error detector
==74322== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==74322== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==74322== Command: ./fsshell SampleVolume 10000000 512
==74322==
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|--------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    OFF   |
| rm                   |    ON    |
| cp                   |    OFF   |
| mv                   |    OFF   |
| cp2fs                |    OFF   |
| cp2l                 |    OFF   |
|--------------------------------|
Prompt > ls

Abha
Deshpande
test1
Prompt > cd Abha
Prompt > ls

test1
Prompt > cd test1
Prompt > ls

stackTest
Prompt > cd stackTest
Prompt > pwd
/Abha/test1/stackTest
Prompt > cd ..
Prompt > pwd
/Abha/test1
Prompt > cd ..
Prompt > pwd
/Abha
Prompt >
```

```
└─ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does not exist, errno = 2
File SampleVolume not good to go, errno = 2
Block size is : 512
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 bytes.
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    ON    |
| rm                   |    ON    |
| cp                   |    ON    |
| mv                   |    ON    |
| cp2fs                |    ON    |
| cp2l                 |    ON    |
|---------------------------------|
Prompt > md hiDir
Prompt > cd ./hiDir/./.././../hiDir/.
Prompt > pwd
/hiDir
Prompt > exit
```

Command: history

```
student@student:~/Documents/csc415-filesystem-CATS$ valgrind --leak-check=full ./fsshell SampleVolume 10000000 512
==74664== Memcheck, a memory error detector
==74664== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==74664== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==74664== Command: ./fsshell SampleVolume 10000000 512
==74664==
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    OFF   |
| rm                   |    ON    |
| cp                   |    OFF   |
| mv                   |    OFF   |
| cp2fs                |    OFF   |
| cp2l                 |    OFF   |
|---------------------------------|
Prompt > ls

Abha
Deshpande
test1
Prompt > cd Abha
Prompt > history
ls
cd Abha
history
Prompt >
```

Command: ls -l

```
student@student:~/Documents/csc415-filesystem-CATS$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    OFF   |
| rm                   |    ON    |
| cp                   |    OFF   |
| mv                   |    OFF   |
| cp2fs                |    OFF   |
| cp2l                 |    OFF   |
|---------------------------------|
Prompt > ls -l

D        16728   Abha
D        16728   Deshpande
D        16728   test1
Prompt >
```

```
| cp2l                 |    ON    |
|---------------------------------|
Prompt > touch hiFile
Prompt > md hiDir
Prompt > ls -l

-            0   hiFile
D        16728   hiDir
Prompt > ls -a

.
..
hiFile
hiDir
Prompt > ls

hiFile
hiDir
Prompt > exit
```

Command: help

```
student@student:~/Documents/csc415-filesystem-CATS$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    OFF   |
| rm                   |    ON    |
| cp                   |    OFF   |
| mv                   |    OFF   |
| cp2fs                |    OFF   |
| cp2l                 |    OFF   |
|---------------------------------|
Prompt > help
ls      Lists the file in a directory
cp      Copies a file - source [dest]
mv      Moves a file - source dest
md      Make a new directory
rm      Removes a file or directory
touch   Touches/Creates a file
cat     Limited version of cat that displace the file to the console
cp2l    Copies a file from the test file system to the linux file system
cp2fs   Copies a file from the Linux file system to the test file system
cd      Changes directory
pwd     Prints the working directory
history Prints out the history
help    Prints out help
Prompt >
```

Command: md

```
student@student:~/Documents/csc415-filesystem-CATS$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    OFF   |
| rm                   |    ON    |
| cp                   |    OFF   |
| mv                   |    OFF   |
| cp2fs                |    OFF   |
| cp2l                 |    OFF   |
|---------------------------------|
Prompt > ls

Abha
Deshpande
test1
Prompt > md TestMd
Prompt > ls

Abha
Deshpande
test1
TestMd
Prompt >
```

Command: rm

```
student@student:~/Documents/csc415-filesystem-CATS$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    OFF   |
| rm                   |    ON    |
| cp                   |    OFF   |
| mv                   |    OFF   |
| cp2fs                |    OFF   |
| cp2l                 |    OFF   |
|---------------------------------|
Prompt > ls

Abha
Deshpande
test1
TestMd
Prompt > rm Deshpande
Prompt > ls

Abha
test1
TestMd
Prompt >
```

```
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 bytes.
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    ON    |
| rm                   |    ON    |
| cp                   |    ON    |
| mv                   |    ON    |
| cp2fs                |    ON    |
| cp2l                 |    ON    |
|---------------------------------|
Prompt > touch hiFile
Prompt > md hiDir
Prompt > rm hiFile
Prompt > ls

hiDir
Prompt > rm hiDir
Prompt > ls

Prompt > exit
```

Command: touch

```
student@student:~/Documents/csc415-filesystem-CATS$ make run
./fsshell SampleVolume 10000000 512
File SampleVolume does exist, errno = 0
File SampleVolume good to go, errno = 0
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    OFF   |
| rm                   |    ON    |
| cp                   |    OFF   |
| mv                   |    OFF   |
| cp2fs                |    OFF   |
| cp2l                 |    OFF   |
|---------------------------------|
Prompt > cd Abha
Prompt > touch Deshpande
Prompt > ls

test1
Deshpande
Prompt > cd Deshpande
Could not change path to Deshpande
Prompt > ls -l

D        16728   test1
-            0   Deshpande
Prompt >
```

Command: cp2fs, cat

```
student@student ~/csc415/csc415-filesystem-CATS (main%) 09:56
└ cat hgello.txt
hgello
```

```
Created a volume with 9999872 bytes, broken into 19531 blocks of 512 bytes.
Opened SampleVolume, Volume Size: 9999872;  BlockSize: 512; Return 0
Initializing File System with 19531 blocks with a block size of 512
|---------------------------------|
|------- Command ------|- Status -|
| ls                   |    ON    |
| cd                   |    ON    |
| md                   |    ON    |
| pwd                  |    ON    |
| touch                |    ON    |
| cat                  |    ON    |
| rm                   |    ON    |
| cp                   |    ON    |
| mv                   |    ON    |
| cp2fs                |    ON    |
| cp2l                 |    ON    |
|---------------------------------|
Prompt > cp2fs hgello.txt hiFs
Prompt > ls

hiFs
Prompt > cat hiFs
hgello
Prompt > exit
System exiting
```

```
student@student ~/csc415/csc415-filesystem-CATS (main%) 09:58
```

Command: cp, mv

```
|---------------------------------|
Prompt > cp2fs hgello.txt hiFs
Prompt > ls

hiFs
Prompt > cat hiFs
hgello
Prompt > cp hiFs hiFsCopy
Prompt > ls

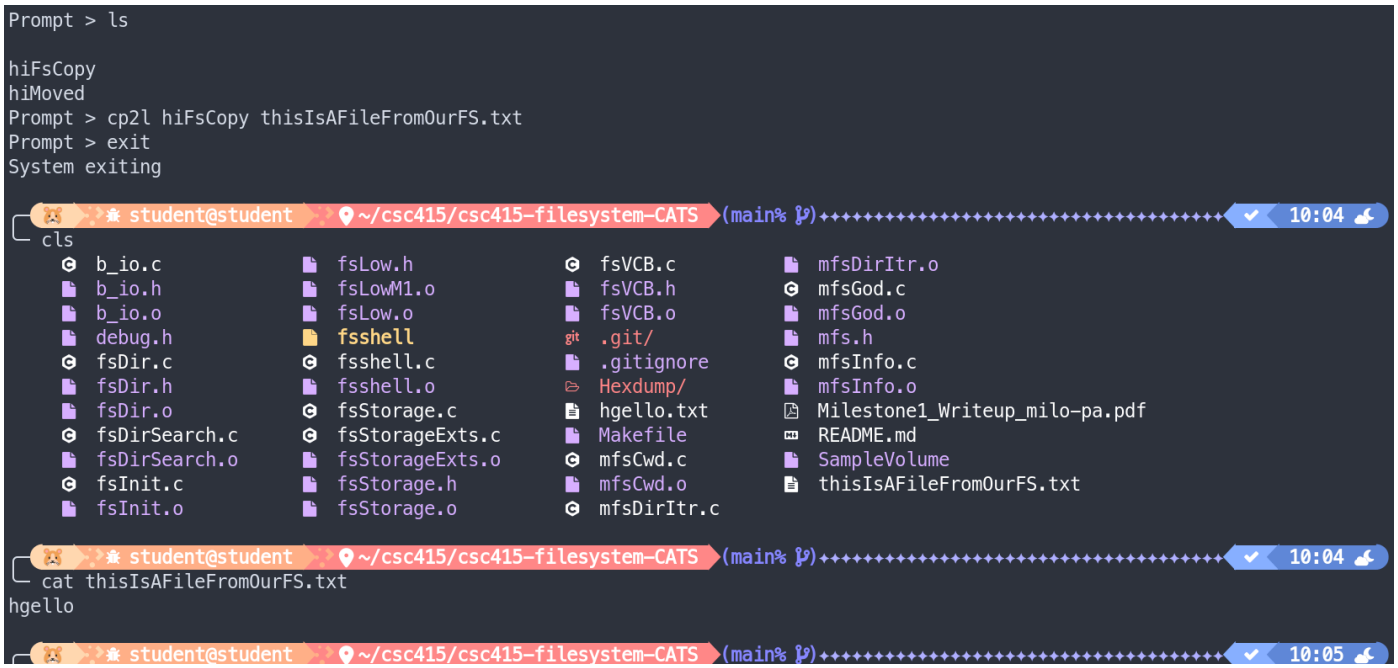
hiFs
hiFsCopy
Prompt > cat hiFsCopy
hgello
Prompt > mv hiFs hiMoved
Prompt > ls

hiFsCopy
hiMoved
Prompt > cat hiMoved
hgello
Prompt > exit
System exiting
```

```
student@student ~/csc415/csc415-filesystem-CATS (main%) 10:01
```

Command: cp2l





Milestones 1 and 2 Write-Ups

VCB Struct Description



The volume control block is the central metadata structure of the file system and is stored at block 0 of our volume. It contains critical information needed to interpret the layout of the volume, functioning as the backbone of the file system by maintaining different core components to ensure that the system can correctly interpret the disk structure, recover its state after restarting, and provide consistency in accessing files and directories.

Our VCB consists of the following 64-bit integer members: blockCount holding the total blocks in our volume, rootLocation holding the location of our root directory, fsLoc holding the location of the start of our freespace extent array (to be initialized with 10 extents). These members are followed by two 32-bit integers: signature, our ‘magic number’ which holds the hex value of our team members’ initials combined (ADAYMPRMTP), and blockSize, the number of bytes per block which is assigned from the passed value through initFileSystem. The sizes of these data types worked out perfectly because the size of our VCB came out to 16 bytes leaving us with no external padding (which we love to see).

During initialization, the VCB is loaded from disk into memory via loadVCB() function. This function allocates memory based on the size of our VCB struct and our static block size, and reads the first block of the volume via LBAread(). The signature member is then read once it is loaded to see if it matches our expected signature. If it does match,the file system assumes that the disk contains valid metadata and skips the formatting process. If it doesn’t, the system initializes the VCB by first setting the block count, size, and signature, followed by initializing its free space and root directory. The free-space global extent array initializes first, from which we store its starting block location in the VCB. The root directory is then created and initialized, again storing its starting block location in VCB. Once all of these are populated, the VCB then is written back to disk using LBAwrite(), ensuring that the file system state is preserved across executions.

Free Space Struct Description



The free space system is responsible for tracking all unallocated blocks on the volume and efficiently providing space for files and directories. We are using a two-level extent table system to track free space. Each extent consists of two 64-bit integers: a location indicating the starting block and a length indicating how many contiguous blocks are available. Extents will be organized into primary and secondary which are created as needed as the disk becomes fragmented and the number of free extents increases. We are not implementing a tertiary extent table as it is not called for in this project's scope.

The free space system is initialized during file system formatting through the initFreeSpace() function. The starting location of the free space extent array is dynamically calculated based on the size of the VCB and block size to ensure that it is placed immediately after the VCB on disk. We allocate a fixed number of blocks (currently 10) to store the extent table. The number of extents that can be stored is determined by how many extent structures fit within those blocks.

At initialization, the system creates a single large extent representing all available space after the reserved regions (VCB and free space blocks). This extent is stored both in memory (via the global pointer freespaceExtsGlobal) and written to disk using LBAwrite(). All remaining entries in the extent table are initialized to zero and act as sentinels to indicate unused entries. A global flag (freeSpaceInitialized) is set to ensure that allocation functions are not used before initialization.

Block allocation is handled by the getBlocks() function, which implements first-fit allocation. The function scans the extent table for available space and selects extents that satisfy the requested number of blocks and a minimum contiguous allocation size. If a large enough extent is found, that extent is allocated and the corresponding free space extent is updated by adjusting its starting location and reducing its length.

The function returns multiple extents if the requested space cannot be satisfied contiguously. These allocated extents are returned as a dynamically allocated array, terminated by a sentinel extent with zeroed fields. Once allocation is complete, the updated free space extent table is written back to disk to ensure persistence across executions.

If insufficient space is available to satisfy the request, the function returns NULL, indicating allocation failure.

Root Directory Description



Our directory system is implemented using a directory_entry structure that stores all essential metadata for files and directories. Each entry contains a 64-bit size field, timestamps (creation, last edit, and last access), an array of extents for block locations (initialized with 5 extents), a 1 byte Boolean indicating whether the entry is a file or directory, and a name field of 255 bytes. Unused entries are marked by setting the first character of the name to null terminator ('\0'), telling us which are free entries. Given these, the total size of directory_entry struct includes 24 bytes of external padding.

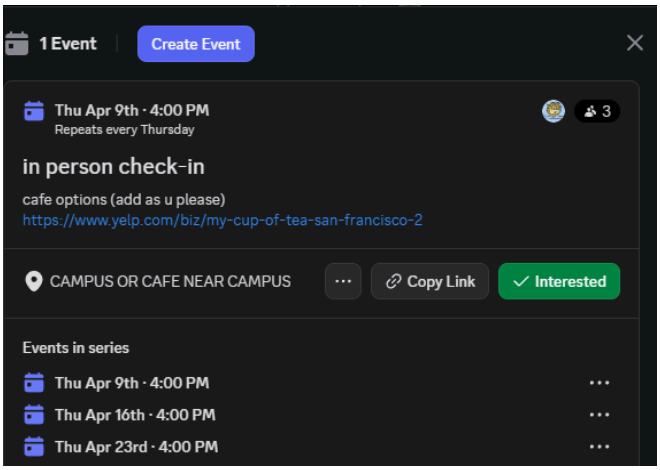
The root directory is created by calling createDirectory() function from fsDir.c. This function allocates memory for the directory based on the requested number of entries, rounding up to a multiple of the block size to ensure compatibility with disk operations. It then requests contiguous blocks from the free-space system and initializes the directory structure in memory before writing it to disk.

For the root directory specifically, special handling is applied since it has no parent. The required "." entry is initialized to reference the root directory itself, including its metadata and extents. The ".." entry, which normally points to the parent directory, is also set to reference the root directory, ensuring consistency with file system conventions.

After initialization, all remaining directory entries are marked as unused with calloc(), then is written to disk using LBAwrite, and its starting block location is stored in the Volume Control Block (VCB). This allows the file system to reliably locate the root directory during future operations.

The root directory plays a critical role in system initialization, as it serves as the entry point for all file system navigation. During formatting in fsInit.c, the free-space system is first initialized, followed by the creation and persistence of the root directory, establishing the foundation for the entire file system hierarchy.

Team Coordination



How often you met
We met twice a week starting the week of the File System Design deadline.

How did you meet
The first meeting was in person and the other was over a voice call, and in between those calls we messaged frequently over our Discord server to check in with our understanding of our progress as we went.

How did you divide up tasks

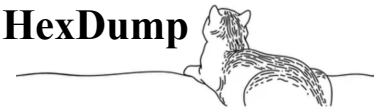
a) File System Design Assignment 1 - Directory entry design
Creating the directory entry struct was done in one session and took a couple hours to complete together. This assignment did not require delegation as it required everyone to be on the same page to put together this foundational struct. The core of the struct was already completed given the previous lectures on what a directory entry should have, and we agreed to add dates for creation and last edited/accessed for the user's convenience. While not everyone could be present for the meet up in the library, we made up for it by bringing everyone up to speed with the reasons behind each variable and data type chosen for our directory entry.

b) File System Design Assignment 2 - VCB and free space prototypes
We divided this assignment into three parts: defining the struct for VCB, outlining the free space design and how it will be tracked, and creating the function signatures for our free space. This work was done together over a voice call on a shared doc so we could check in with each other's work and make sure the pieces were fitting together. VCB was outlined by Allie, Rosa defined extent and extent_table structs (deprecated the latter), Trish and Abha worked on the function prototypes for our extents and VCB, and Milo bounced between helping Allie and making sure the rest followed this schema

c) File System Milestone 1 - Implementing Assignments 1 and 2
This milestone took about 2 weeks to really get pieced together, starting with a meeting at a cafe (see the Discord calendar screenshot above) Here, since we had the File System Design assignment to work off of, the first step was to delegate who would actually implement our VCB

struct and what functions would be needed to interact with this struct. In this meeting, Allie handled practically all of the header and C files for this with Milo “back-seat coding”. Abha started implementing the free space header and functions, Trish set up our CATS-tastic write-up, and Rosa scaffolded the initialization function for our file system. It took us a while to figure out what was going on, even with the Steps document, but that helped immensely in divvying up the work. After this meeting, we agreed to communicate often in Discord before attempting to work on a file or pushing to Git. From there, since Allie and Milo worked on the central part of the milestone, they delegated various helper functions and methods to the others to finish implementations for fsInit().

HexDump



NOTE: Running lscpu on the VM shows that the byte order on the machine is little endian, meaning the least significant bytes (on the right side) are stored at lower memory addresses. This is why they appear “backwards” in the hexdump.

```
 student@student  ~/csc415/csc415-assignment-2-bufferandstruct-flimsychip (main*%)++++++++++++  09:38
└─ lscpu
Architecture:                x86_64
  CPU op-mode(s):            32-bit, 64-bit
  Address sizes:             45 bits physical, 48 bits virtual
  Byte Order:                Little Endian
```

BLOCK 0 (Volume Control Block)

```
000200: 4B 4C 00 00 00 00 00 00  0B 00 00 00 00 00 00 00 | KL**************
000210: 01 00 00 00 00 00 00 00  77 0F DC 8A 00 02 00 00 | ********w*??****
000220: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000230: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000240: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000250: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000260: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000270: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000280: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000290: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0002F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

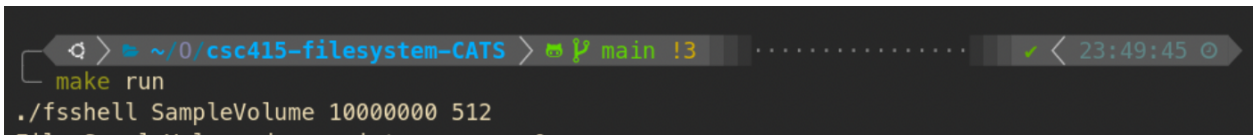
000300: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000310: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000320: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000330: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000340: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000350: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000360: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000370: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

```
000380: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000390: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0003F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

Bytes in this color are all unused bytes to fill the remaining block space.

HexDump Analysis:

- ➔ Value: `4B 4C 00 00 00 00 00 00`
- ➔ Address:`000200`
- ➔ Type: `uint64_t`
- ➔ Explanation: This is the blockCount which shows the total number of blocks in the volume. In little endian, this is read as 0x4C4B which is equal to 19531 blocks, which is the same as our volume size of 10000000 bytes / 512 (blockSize), rounded down = 19531 bytes per block. This volume size and block size confirmed by the arguments passed to make run



- ➔ Value: `0B 00 00 00 00 00 00 00`
- ➔ Address: `000208`
- ➔ Type:`uint64_t`
- ➔ Explanation: This is the rootLocation. This is the block number where the root directory lives on our simulated disk. In little endian, `0X0B` translates to 11 which is where the filesystem will look first when navigating through the directory tree. This is the block number where the root director lives, as we confirmed by printing the value and by our analysis of blocks 11 and 12 further down

- ➔ Value:`01 00 00 00 00 00 00 00`
- ➔ Address:`000210`
- ➔ Type:`uint64_t`

- ➔ Explanation:This shows the freeSpaceLocation. In little endian, this is equivalent to `0x01 = 1`, which describes the blockNumber where the free space extent table begins.

- ➔ Value:`77 0F DC 8A`
- ➔ Address:`000218`
- ➔ Type:`uint32_t`
- ➔ Explanation:This shows the signature for our filesystem. The Little-endian for this is: `0x8ADC0F77 = 2329677687` which matches the number we had hardcoded as our system signature — it's all of our initials translated to phone number digits: AD, AY, MP, RM, TP

- ➔ Value:`00 02 00 00`
- ➔ Address:`00021C`
- ➔ Type:`uint32_t`
- ➔ Explanation:This shows the blockSize. In little endian, this should be `0x0200` which corresponds to the value 512, the number of bytes per block.

BLOCK 1

```
000400: 0D 00 00 00 00 00 00 00  3E 4C 00 00 00 00 00 00 | ********>L******
000410: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000420: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000430: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000440: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000450: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000460: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000470: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000480: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000490: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0004F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000500: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000510: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000520: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000530: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000540: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000550: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000560: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000570: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000580: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000590: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0005F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

- ➔ Value:`0D 00 00 00 00 00 00 00`
- ➔ Address:`000400`
- ➔ Type:`uint64_t`
- ➔ Explanation:This is the extent[0].location. The starting block number of the first free extent. 0x0D means 11 in decimal, which means that the free space begins at block 11. This is confirmed by the fact that our reserved blocks are 1-10 and the root directory takes up two blocks from 11-12, therefore the free space starts at 13. These addresses are confirmed by analysis further down

- ➔ Value:`3E 4C 00 00 00 00 00 00`
- ➔ Address:`000408`
- ➔ Type:`uint64_t`
- ➔ Explanation:This is extent[0].length. This shows the number of free blocks to this extent. 0X4C3E would translate to 19518 in decimal which means there are 19518 contiguous blocks of free space ready to use. As we stated above, we have 19531 blocks in our volume. 19531 - 13 = 19518

- ➔ Value:`00 00 00 00 00 00 00 00` (repeated)
- ➔ Address:`000410` onwards
- ➔ Type:`uint64_t` pairs
- ➔ Explanation: These all contain extent[1] through extent[n]. They all contain zeroes showing that there are no additional extents yet. 0 shows that the free slot is yet to be used.

BLOCK 2

```
000600: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000610: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000620: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000630: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000640: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000650: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000660: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000670: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000680: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000690: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0006F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000700: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000710: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000720: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000730: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000740: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000750: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000760: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000770: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000780: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000790: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0007F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 3

```
000800: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000810: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000820: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000830: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000840: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000850: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000860: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000870: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000880: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000890: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0008F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000900: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000910: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000920: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000930: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000940: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000950: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000960: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000970: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000980: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000990: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0009F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 4

```
000A00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000A90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000AF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000B00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000B90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000BF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 5

```
000C00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000C90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000CF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000D00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000D90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000DF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 6

```
000E00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000E90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000ED0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000EF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

000F00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000F90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
000FF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 7

```
001000: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001010: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001020: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001030: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001040: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001050: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001060: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001070: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001080: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001090: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0010F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001100: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001110: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001120: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001130: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001140: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001150: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001160: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001170: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001180: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001190: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0011F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 8

```
001200: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001210: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001220: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001230: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001240: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001250: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001260: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001270: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001280: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001290: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0012F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001300: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001310: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001320: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001330: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001340: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001350: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001360: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001370: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001380: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001390: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0013F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 9

```
001400: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001410: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001420: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001430: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001440: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001450: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001460: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001470: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001480: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001490: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0014F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001500: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001510: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001520: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001530: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001540: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001550: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001560: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001570: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001580: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001590: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0015F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 10

```
001600: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001610: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001620: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001630: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001640: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001650: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001660: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001670: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001680: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001690: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0016F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001700: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001710: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001720: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001730: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001740: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001750: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001760: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001770: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001780: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001790: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0017F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

BLOCK 11 (Root Directory)

```
001800: E0 02 00 00 00 00 00 00  EA 3B D7 69 00 00 00 00 | ?*******?;?i****
001810: EA 3B D7 69 00 00 00 00  EA 3B D7 69 00 00 00 00 | ?;?i****?;?i****
001820: 0B 00 00 00 00 00 00 00  00 02 00 00 00 00 00 00 | ****************
001830: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001840: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001850: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001860: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001870: 01 2E 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | *.**************
001880: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001890: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018E0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0018F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001900: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001910: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001920: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001930: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001940: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001950: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001960: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001970: E0 02 00 00 00 00 00 00  EA 3B D7 69 00 00 00 00 | ?*******?;?i****
001980: EA 3B D7 69 00 00 00 00  EA 3B D7 69 00 00 00 00 | ?;?i****?;?i****
001990: 0B 00 00 00 00 00 00 00  02 00 00 00 00 00 00 00 | ****************
0019A0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0019B0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0019C0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0019D0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
0019E0: 01 2E 2E 00 00 00 00 00  00 00 00 00 00 00 00 00 | *..*************
0019F0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

- ➔ Value : `E0 02 00 00 00 00 00 00`
- ➔ Address: `001800`
- ➔ Type: `uint64_t`

- ➔ Explanation: This is the size of the directory entry measured in bytes. 0X02E0 in little endian which is the same as 736 bytes. So size = 736 bytes is the total space used by the directory entries. This is because the directory entry was created with the initialEntryCount = 2. Because of the following line of code in fsInit:
  directory_entry *root = createDirectory(2, NULL);

- ➔ Value:`EA 3B D7 69 00 00 00 00`
- ➔ Address:`001808`
- ➔ Type:`time_t`
- ➔ Explanation:This is the creationDate which is of time_t, which typically takes up 8 bytes. To check what date this corresponds to, this is still to be converted to little endian, which turns into `0X69D73BEA` = 1,775,929,322 seconds, since January 1, 1970 which corresponds to approximately April 2026.

- ➔ Value :`EA 3B D7 69 00 00 00 00`
- ➔ Address:`001810`
- ➔ Type:`time_t`
- ➔ Explanation:This is the lastEditDate, the same value as the creationDate. The value is the same, and repeated for lastAccessDate as well since the directory was just created.

- ➔ Value:`EA 3B D7 69 00 00 00 00`
- ➔ Address:`001818`
- ➔ Type:`time_t`
- ➔ Explanation:This is the lastAccessDate, the same value as the creationDate as well.

- ➔ Value:`0B 00 00 00 00 00 00 00`
- ➔ Address:`001820`
- ➔ Type:`uint64_t`
- ➔ Explanation:This shows extent[0].location which is the starting block number for where the root directory data is stored on the disk. 0x0B is 11 in decimal which is indeed the location in which the root directory lives, block 11.

- ➔ Value:`00 02 00 00 00 00 00 00`
- ➔ Address:001828
- ➔ Type:uint64_t

- ➔ Explanation: This shows extent[0].length. 0x0400 in little endian translates to 1024 which is the total size of the root directory data on the disk. 1024 is the total bytes of data allocated on disk for the root directory

- ➔ Value :`00 00 00 00 00 00 00 00`
- ➔ Address:`001830`
- ➔ Type:`uint64_t`
- ➔ Explanation:This shows extent[1] to extent[4] which are the remaining 4 unused exts in the array. The root directory only needs one extent to store its data, the rest of the extents are zeroed out.

- ➔ Value:`01`
- ➔ Address:`001870`
- ➔ Type:`bool`
- ➔ Explanation:This shows the entryType in the directory. 0 corresponds to a file and 1 corresponds to a directory. Since the value is 1, it means that it's a directory.

- ➔ Value:`00 00 00 00 00 00 00 00` (repeated)
- ➔ Address: `001873 - 00196F`
- ➔ Type:
- ➔ Explanation: This is all the unused space in the 255 byte array for the directory entry name.

- ➔ Value: `01`
- ➔ Address: `0019E0`
- ➔ Type: `bool`
- ➔ Explanation: This shows the entryType in the directory. 0 corresponds to a file and 1 corresponds to a directory. Since the value is 1, it means that it's a directory.

- ➔ Value: `2E 2E`
- ➔ Address: `0019E1 - 0019E2`
- ➔ Type: `char`
- ➔ Explanation: This corresponds to the name of the second directory entry. The ASCII for 2E 2E corresponds to "..", which means the current directory.

- ➔ Value: `00`
- ➔ Address: `0019E3`
- ➔ Type: `char`
- ➔ Explanation: This is a null terminator('\0'), right after the name of the directory "..".

The following screenshot has some debug statements used to check whether the values in the hexdump are correct.

```
[DEBUG] fsFreeSpace.c:73: freespaceLocation: 1
[DEBUG] fsFreeSpace.c:75: firstFreeBlock: 11
[DEBUG] fsFreeSpace.c:77: extents[0].location: 11
[DEBUG] fsFreeSpace.c:79: extents[0].length: 19520
[DEBUG] fsFreeSpace.c:81: numExtents: 320
[DEBUG] fsInit.c:72: Free space initialized successfully
[DEBUG] fsDir.c:49: Creating directory with 2 entries, which requires 1024 bytes and 2 blocks
.
[DEBUG] fsInit.c:93: Root directory created successfully
[DEBUG] fsInit.c:104: VCB written to disk successfully. File system initialized
```

BLOCK 12

```
001A00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001A90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001AF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************

001B00: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B10: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B20: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B30: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B40: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B50: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B60: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B70: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B80: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001B90: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BA0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BB0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BC0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BD0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BE0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
001BF0: 00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 | ****************
```

- ➔ Value: `00 00 00 00 00 00 00 00` (repeated)
- ➔ Address:
- ➔ Type:
- ➔ Explanation: This is padding/free space, not currently occupied.